

Celaya NFC 控制器 基于 HiDiTingV100 SDK

开发指南

文档版本 01

发布日期 2025-06-27

前言

概述

本文档详细的描述了 NFC SDK 包中 NFC（Near Field Communication）控制器芯片的开发、使用方法，主要包括简介、NFC 系统架构、NFC 驱动说明、NFC 控制器软件说及测试方法和常见问题。

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
 危险	表示如不避免则将会导致死亡或严重伤害的具有高等级风险的危害。
 警告	表示如不避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不避免则可能导致轻微或中度伤害的具有低等级风险的危害。
 须知	用于传递设备或环境安全警示信息。如不避免则可能会导致设备

符号	说明
	损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
01	2025-06-27	第一次临时版本发布。

目 录

前言.....	i
1 NFC 软件分层框图.....	1
2 NFC 驱动说明.....	2
2.1 驱动描述.....	2
2.2 驱动源码说明.....	2
2.2.1 NFC 相关驱动接口.....	3
2.3 NFC 固件加载说明.....	4
2.4 编译适配.....	4
2.5 nfc_hal_adapter.h 驱动适配层接口.....	4
3 NFC 软件说明及测试方法.....	6
3.1 NFC 源码说明.....	6
3.2 产品的 HAL 层接口函数.....	8
3.3 编译流程.....	10
3.4 编译宏说明.....	10
3.5 版本烧录.....	10
3.6 NFC 功能测试.....	10
4 常见问题.....	13
4.1 固件加载失败.....	13
4.2 NFCEE 激活失败.....	13
4.3 I2C 通路异常.....	13
A 缩略语.....	15

插图目录

图 1-1 NFC 软件分层图 1

表格目录

表 2-1 驱动代码文件路径说明..... 2

表 2-2 NFC 相关驱动接口说明 3

表 2-3 nfc_hal_adapter.h 接口函数说明 5

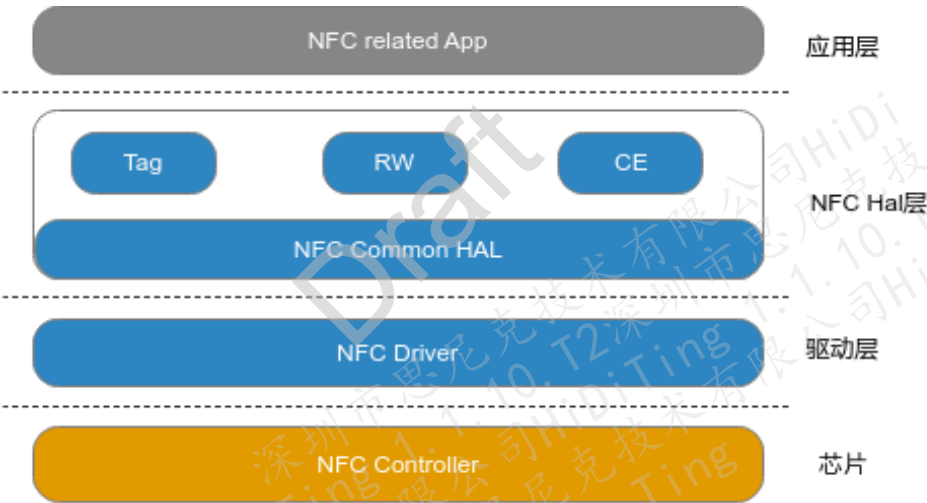
表 3-1 HAL 层定制化参数修改..... 8

表 3-2 NFC 功能编译宏 10

1 NFC 软件分层框图

嵌入式系统上的 NFC 软件分层图如图 1-1 所示。

图1-1 NFC 软件分层图



RTOS 嵌入式系统上的 NFC 软件分层图说明如下：

- 框图颜色说明：灰色为用户自有模块，蓝色为本方案软件，橙色为本方案硬件。
- NFC Driver 是 NFC 芯片的驱动，用于访问底层 NFC 控制器资源。
- NFC HAL 是硬件抽象层，用于向上层提供硬件服务的抽象接口。具体分为通用的 Common HAL 部分和功能模块 RW/CE/TAG 接口代码。
- 上层 NFC 相关的 APP 直接调用 HAL（Hardware Abstract Layer）层的接口处理业务。

2 NFC 驱动说明

- 2.1 驱动描述
- 2.2 驱动源码说明
- 2.3 NFC 固件加载说明
- 2.4 编译适配
- 2.5 nfc_hal_adapter.h 驱动适配层接口

2.1 驱动描述

NFC 驱动通过 I2C 和底层的 NFC 控制器通讯。主要实现 NFC 芯片固件加载、NFC 上电启动、给上层提供通用的接口用于访问底层 NFC 控制器资源。

2.2 驱动源码说明

NFC SDK 包中 NFC 所用驱动代码位于 “middleware/services/srv_tiot_host”，以源码形式提供，主要包括 common、device、product_porting 三个部分。

驱动需要依赖的板级硬件接口主要有四组：I2C 数据收发、GPIO 控制、日志打印、板级硬件信息配置。本质上，板级硬件接口移植适配是将不同用户自身的硬件控制接口转换为本驱动需要依赖的固定接口，具体实现不在此赘述，以下只对文件路径及 NFC 使用的相关接口进行简单说明。

表2-1 驱动代码文件路径说明

目录	子文件夹和文件的说明
----	------------

目录	子文件夹和文件的说明
common(dir)	驱动主体功能代码（不需要改动）。
product_porting(dir)	产品相关适配，主要用来实现 OS 适配层和板级适配层。需要移植修改。NFC SDK 代码包中提供了相关 sample，用户适配时可以参考。
device(dir)	器件相关适配代码，实现固件加载相关功能，NFC 芯片固件也保存在此目录下。

2.2.1 NFC 相关驱动接口

以下接口主要实现 MCU 与 NFC 芯片之间的硬件接口操作，如 GPIO 控制、I2C 读写等，在 NFC SDK 包中已通过驱动适配层封装成用户上层直接可以使用的接口，在“2.5 nfc_hal_adapter.h 驱动适配层接口”中会具体进行说明。

表2-2 NFC 相关驱动接口说明

函数	功能说明
int32_t tiot_service_init(void *param)	板级初始化，主要是 GPIO、数据结构等的初始化，tiot 只能初始化一次，若已初始化过且没有去初始化过，再调用该接口会执行失败。
void tiot_service_deinit(void)	驱动去初始化，仅在驱动初始化成功后可以调用。
tiot_handle tiot_service_open(const char *device_name, void *param)	打开 tiot 服务，完成 NFC 固件加载。
void tiot_service_close(tiot_handle handle)	关闭 tiot 服务。
int32_t tiot_service_write(tiot_handle handle, const uint8_t *data, uint32_t len)	通过 I2C 往 NFC 芯片写数据。
int32_t tiot_service_read(tiot_handle handle, uint8_t *data, uint32_t buff_len, uint32_t timeout_ms)	通过 I2C 从 NFC 芯片读数据，timeout_ms 表示等待读超时时间，单位为 ms，最大值根据平台有所差异，用户可以根据实际情况适配。

2.3 NFC 固件加载说明

目前提供的 SDK 中 NFC 可以通过大数组或文件读取方式加载固件，通过编译宏 CONFIG_FILE_BY_ARRAY 控制，默认采用文件读取方式加载。两种方式使用的固件文件和配套驱动有少量差异，以下为详细说明：

- 若采用大数组方式加载，需要关注 NFC 固件文件 “caxx_cfg.bin”，在 NFC SDK 包中该固件文件位于
“middleware/services/srv_tiot_host/tiot_driver/device/caxx/firmware” 路径下，如有固件更新，从发布的固件包中替换掉这个路径下的固件文件及配套的
“caxx_fw_file_array.c 和 caxx_fw_file_array.h” 再重新编译即可。
- 若采用文件加载，需要关注
“middleware/services/srv_tiot_host/tiot_driver/product_porting/\$(product_name)/caxx_board_port_config.h” 文件中固件路径 CAXX_FW_CFG_PATH 的定义，需要将 NFC 固件文件 “fw.img” 推入文件系统后，修改该路径宏并重新编译。

2.4 编译适配

NFC SDK 包使用 CMake 进行编译，添加了 Kconfig 可以灵活的设置一些编译选项，用户拿到手的 SDK 包中已默认将该驱动添加到编译中，用户可以直接编译使用，若需要修改文件加载方式，可在 Kconfig 中修改 use array file 这一编译选项。

2.5 nfc_hal_adapter.h 驱动适配层接口

上层访问驱动的接口都在 “nfc_hal_adapter.h” 中定义，上层代码只需要包含这个头文件，即可访问驱动的所有功能，建议不要调用驱动其它文件的接口。当前定义了五个接口，一般不需要改动，用户也可以根据需要定制。NFC 的驱动接口原则上只能由 NFC HAL 层代码调用。

值得注意的是目前 NFC SDK 中 NFC 和 GNSS 共用 tiot 驱动，此时驱动的初始化和去初始化操作可能会产生冲突，如有必要可以自行修改 HW_NFC_Open 和 HW_NFC_Close 中对驱动初始化和去初始化接口的调用，提出来作为单独的接口使用即可。

表2-3 nfc_hal_adapter.h 接口函数说明

函数	功能说明
int HW_NFC_Open(void)	打开 NFC，驱动初始化，NFC 芯片上电，进行固件加载。
int HW_NFC_Close(void)	关闭 NFC，NFC 芯片下电，驱动去初始化。
int HW_NFC_Msleep(unsigned int msec)	睡眠函数，单位为 ms。
int HW_NFC_Write (uint8_t *buf, uint16_t count)	往 NFC 芯片写数据。
int HW_NFC_Read (uint8_t *buf, uint16_t bufLen, unsigned long timeout)	从 NFC 芯片读数据，如果没有数据会阻塞，直到 timeout 后触发超时机制结束本次读取，timeout 单位为 ms，可设置范围与平台相关，请用户根据实际情况适配。

3 NFC 软件说明及测试方法

本章节将对 NFC SDK 软件包中提供的 NFC 软件业务代码进行详细说明。

3.1 NFC 源码说明

3.2 产品的 HAL 层接口函数

3.3 编译流程

3.4 编译宏说明

3.5 版本烧录

3.6 NFC 功能测试

3.1 NFC 源码说明

在 NFC SDK 提供的软件包中，NFC 代码位于 “middleware/services/nfc” 目录下，头文件位于 “include/middleware/services/nfc” 目录下，sample 位于 “application/samples/nfc” 目录下。

NFC 代码结构如下所示：

```
$(root)
├─middleware
│   └─$(product_name)
│       └─services
│           └─nfc
│               └─hal                    核心HAL层代码
│                   └─core                HAL层通用代码文件夹
```

				└hal_adapter	HAL驱动适配层代码文件夹
				└mxxce	mxx卡模拟模式操作函数库
				└tag	
				...	读卡器对Tag协议操作函数库
				└ndef	
				...	读卡器对Ndef协议操作函数库
				└srv_tiot_host	tiot驱动代码
				└tiotd_driver/device/caxx/firmware	
				...	大数组形式的固件和路径cfg文件
				└include	
				└middleware	
				└services	
				└nfc	NFC相关头文件
				└application	演示接口调用的参考文件文件夹
				└samples	
				└nfc	
				└controller	
				└example_common	读卡器、卡模拟、MXX卡模拟和NFC标签样例代
					码
				example_ce.c	
				example_ce.h	
				example_rw.c	
				example_rw.h	
				example_tag.c	
				example_tag.h	

```
|      example_mxxce.c
|
|      example_mxxce.h
|
|
└─sample_nfc_only
...      只包含nfc功能的示例代码
```

关于 HAL 层 API 介绍，请参见相关的《NFC RTOS SDK API》 doxygen 说明文档。同时可以参考 sample 文件的实现内容，了解 HAL 层接口函数的调用方法。

3.2 产品的 HAL 层接口函数

“\$(Linux_root)/vendor/nfc/hal/samples/controller/example_common/” 下面有 example_rw.c、example_ce.c、example_tag.c 和 example_mxxce.c 四个功能示例代码，用户可以参照示例代码中 NFC_EXAMPLE_RwInit、NFC_EXAMPLE_CeInit、NFC_EXAMPLE_TagInit 和 NFC_EXAMPLE_StartMxxCardEmulation 展示的流程，根据产品进行参数或代码微调，需要调整的地方包括但不限于表 3-1 所示内容。

表3-1 HAL 层定制化参数修改

函数	描述	RW 模式	CE 模式	TAG 模式
NFC_HAL_SetCoreParams	设置 HAL 层核心参数。	NA	默认配置： 1. NCI 控制包读操作超时时间 1000ms。 2. NCI 数据包读操作超时时间 1500ms。 3. 处理 NFCEE-HCI 消息的回调函数。	NA
NFC_HAL_NciSetConfig	配置 CORE_SE T_CONFIG_CMD 命	默认配置： 配置参数 1： 0xA003，关闭状态	默认配置： 配置参数 1： 0xA300，nfcee	默认配置： 配置参数 1： 0x80，

函数	描述	RW 模式	CE 模式	TAG 模式
	令。	保护定时器。 配置参数 2: 0x1F4, 轮询周期内 listen 时间, 默认值 500ms。 配置参数 3: NFCC_CONFIG_CONTROL, 允许 NFCC 整合 RF 参数。 配置参数 4: CON_DISCOVERY_PARAM, 允许 Poll 模式。	使能标志位, 默认使能 eSE, 产品根据芯片类型和使用场景来配置。	使能场检测上报通知。
NFC_HAL_CfgLowPwr	配置 device 低功耗模式。	默认开启低功耗模式。	默认开启低功耗模式。	默认开启低功耗模式。
NFC_HAL_NciDiscovMap	配置 RF_DISCOVER_MAP_CMD 命令。	默认配置: IsoDep 协议映射到 IsoDep 接口。	默认配置: IsoDep 协议映射到 IsoDep 接口。	NA
NFC_HAL_StartDiscovery	配置 RF_DISCOVER_CMD 命令。	默认配置: 支持 ABFV 四种技术。	默认配置: 支持 A 技术。	默认配置: 支持 A 技术 (Tag 模式默认只支持 A 技术的 T2T)。

3.3 编译流程

NFC SDK 使用 CMake 进行编译构建、相关代码库中应该都包含相应的 CMakeLists 文件，添加了 Kconfig 可以灵活的设置一些编译选项，用户拿到手的 SDK 包中应该已将该 NFC 相关源码添加到编译中，请注意，目前提供的版本里，HAL 层 RW（Reader/Writer，读卡器）和 CE（Card Emulation，卡模拟）功能均默认打开，若用户需要只使用其中部分功能，可以在 build 仓中“config.py”中通过 CONFIG_HAVE_NFC_POLL 和 CONFIG_HAVE_NFC_LISTEN 调整功能使能编译宏 HAVE_NFC_POLL、HAVE_NFC_LISTEN。

3.4 编译宏说明

用户编译 NFC 组件时根据功能需要可以自行调整表 3-2 中编译宏，调整方法请参考“3.3 编译流程”。

表3-2 NFC 功能编译宏

编译宏	功能
HAVE_NFC_POLL	NFC 开启读写器功能。
HAVE_NFC_LISTEN	NFC 开启卡模拟功能。

3.5 版本烧录

将编译输出的结果烧录到产品中并测试系统是否加载正常，用户可以通过日志打印等方式进行确认，具体测试流程可参考“3.6 NFC 功能测试”。

3.6 NFC 功能测试

当前 SDK 版本中 NFC 不会在开机时默认加载，而是通过 AT 指令控制开启、关闭，目前支持的 AT 指令有以下几条：

- AT+NFCREBOOT：重启 NFC，完成 NFC 固件加载。
- AT+NFCRESET：执行 NFC 复位及初始化流程。

- AT^NFCBUSINESSSTART: 开启 NFC 业务, 入参表示开启模式 (0: MXX 卡模拟, 1: 读卡器)。
- AT^NFCBUSINESSSTOP: 停止 NFC 业务。
- AT^NFCMXXCESWITCHCARD: MXX 卡模拟模式下切换 NFC MXX 卡, 入参为切换的目标卡片编号, 执行该指令前请先停止 NFC 业务, 切卡完成后再重新开启。
- AT^NFCOPENLOG: 开启 NFC 芯片日志, 执行该指令前请先停止 NFC 业务, 执行完成后再重新开启。
- AT^NFCCLOSELOG: 关闭 NFC 芯片日志, 执行该指令前请先停止 NFC 业务, 执行完成后再重新开启。
- AT^NFCSENDNCICMD: 发送一条 NCI 命令给 NFC 芯片, 可以用于芯片功能或射频参数的配置, 执行该指令前请先停止 NFC 业务, 执行完成后再重新开启。

基本功能可以按照以下流程进行测试:

步骤 1 加载 NFC 固件: AT^NFCREBOOT。

步骤 2 NFC 初始化及复位: AT^NFCRESET。

步骤 3 开启业务 NFC 业务: “AT^NFCBUSINESSSTART=0” 开启 MXX 卡模拟业务, 或 “AT^NFCBUSINESSSTART=1” 开启读卡器业务。

步骤 4 根据所开启的业务进行刷卡读卡等场景测试。

----结束

用户可以根据日志或者直接刷卡判断业务是否正常。

4 常见问题

5.1 固件加载失败

5.2 NFCEE 激活失败

5.3 I2C 通路异常

4.1 固件加载失败

日志中出现如下信息，表明固件加载有问题，NFC 控制器的固件没有加载成功。请查看固件 FW 存放位置以及文件名和内容是否正确。

```
[TloT][fw]command execute err!
```

4.2 NFCEE 激活失败

NFCEE (UICC) 的激活失败可能由如下原因导致：

- SWP 的 S1 电压存在抖动，会影响 NFC 控制器解析 SIM 卡响应的数据，优先排查 SWP 的硬件器件。
- 部分 SIM 卡除了 SWP 的接口外，还需要 Host 对 7816 接口激活，NFCEE 才能激活成功。

4.3 I2C 通路异常

I2C 通路异常通常为如下原因：

- 焊接异常，版本可以下载成功，业务初始化出现 Reset 消息失败。
- Host 为单核，处理 I2C 消息的优先级比较低，导致 I2C 数据未能及时被 Host 读取，NFC 无法释放业务资源，会产生硬复位，Host 需要对 NFC 重新进行初始化。

Draft

A 缩略语

C		
CE	Card Emulation	卡模拟
H		
HAL	Hardware Abstract Layer	硬件抽象层
M		
MXX	MIFARE	指 NXP 开发的一种加密卡类型
N		
NFC	Near Field Communication	近场通信
NCI	NFC Controller Interface	NFC 控制器接口
R		
RW	Reader/Writer	读写器